Here is a simple, student-friendly comparison between your project workflow and each of the four research papers. I have highlighted the exact differences and explained **your project's novelty** (what makes your project unique and better) compared to each one.

1. "Active Face Liveness Detection Using Mediapipe.pdf"

Workflow Comparison: Their workflow is highly similar to your **ONLINE Active** step. They capture video, use MediaPipe to track facial landmarks, and use the exact Eye Aspect Ratio (EAR) math formula you plan to use to detect blinks. **The Differences:** They do not have an offline dataset or a passive CNN to check skin texture. To detect head movements, they recorded coordinates and trained a separate machine learning model (Random Forest Classifier). **Your Project's Novelty (Why yours is better):**

- **The Fusion Engine:** Their system can be tricked by a high-quality video replay because they only check for movement. Your novelty is adding the **Passive CNN** layer to check the texture of the pixels, making your system much harder to fool.
- **Lighter Logic:** You do not need to train an extra Random Forest model just to see if a head turns; your MediaPipe logic handles this natively, making your browser app much lighter and faster.

---

2. "Art - 3534.pdf" (Face Liveness Detection and Facial Recognition)

Workflow Comparison: This paper's workflow is incredibly close to your **Full System Workflow**. They capture real-time video directly from the browser using HTML5 getUserMedia(). They also use a "Fusion" approach: they use MediaPipe FaceMesh to actively ask the user to blink or smile, while simultaneously using a CNN to passively look at the face texture to spot fake photos. **The Differences:** They use a heavier CNN called ResNet-18 for liveness. More importantly, after the liveness check, they pass the image into another heavy AI called "FaceNet" to recognize exactly *who* the user is (matching the face to a database). **Your Project's Novelty (Why yours is better):**

- **Web-Optimized & Pure Liveness:** Their use of ResNet-18 and FaceNet is very heavy for a web browser. Your novelty is stripping away the heavy "facial recognition" part and focusing purely on a lightweight, ONNX/TFJS-optimized liveness engine. Your project will run significantly faster and smoother on weak client devices (like cheap mobile phones) without crashing the browser tab.

---

3. "Real-Time Optimization and Lightweight Architecture of Face Detection Models..."

Workflow Comparison: This paper matches your **OFFLINE Workflow**. They take a model (MobileNetV1), strip out heavy layers to make it "SlimLite", and specifically convert it to **ONNX format**. They proved that ONNX conversion makes the AI run blazingly fast on weak hardware like a Raspberry Pi. **The Differences:** This paper only builds a Face *Detection* model (finding where the face is in the image). It does not check if the face is a real person (Liveness). It also has no WebRTC, no MediaPipe, and no active user checks. **Your Project's Novelty (Why yours is better):**

- **Applied to Anti-Spoofing:** You are taking their brilliant offline techniques (MobileNet + ONNX conversion for extreme speed) and applying them to a completely different problem: **Face Liveness Classification**.

- **Browser Implementation:** Instead of just running on a standalone Raspberry Pi, you are bringing these optimized ONNX/TFJS models to the web using WebRTC, making it accessible to anyone with a URL.

---

4. "getPDF.jsp.pdf" (Liveness Detection Based on Eye Blinks and Pose Estimation...)

Workflow Comparison: This paper is similar to your **ONLINE Active** step. They use MediaPipe and calculate the Eye Aspect Ratio (EAR) to force the user to blink 5 times before the camera takes a selfie. **The Differences:** Because they do not have a passive CNN, they force the user to prove they are 3D by performing complex arm poses. The system uses trigonometry to measure the exact angles of the user's elbows and shoulders. **Your Project's Novelty (Why yours is better):**

- **Better User Experience (UX):** Forcing a user to step back and raise their arms to specific angles just to log in is a terrible user experience. Your novelty is replacing these annoying arm poses with your silent **Passive CNN**.
- **Better Security:** In their testing, their system failed and allowed a fake login when the attacker wore a paper mask with eye holes cut out (because the attacker could still blink behind the mask). Your Fusion Engine solves this! While the attacker might pass your active blink check, your passive CNN would immediately detect the paper texture of the mask and block the attack.

++++++++++++++++++++++++++++++
+_____
_____
_____
_____

Here is a complete comparison of your project's workflow against **each of the 12 research papers** provided in your sources. I have explained how their workflows compare to yours, the exact differences, and **your project's novelty** (what makes your project unique, better, or more modern) for every single paper in simple words.

---

1. "Active Face Liveness Detection Using Mediapipe.pdf"

- **Workflow Comparison:** This paper's workflow matches your **ONLINE Active** step perfectly. They capture video, use MediaPipe to track facial landmarks, and calculate the Eye Aspect Ratio (EAR) to detect blinks,.
- **The Differences:** They do not have a passive CNN to check skin texture. Also, to detect head movements, they train a separate machine learning model (Random Forest). Your project runs in a web browser, while theirs is built in Python.
- **Your Project's Novelty: The Fusion Engine.** Because they only use active checks, a hacker could hold up a high-quality video of someone blinking to fool their system. Your novelty is adding the **Passive CNN** to look at the pixel texture, which blocks video replay attacks.

2. "Art - 3532.pdf" (Passive Face Liveliness Detection)

- **Workflow Comparison:** This paper matches your **Passive CNN** step. They capture frames and pass them through a Convolutional Neural Network (CNN) to automatically score whether the face is live or spoofed without the user doing anything,.
- **The Differences:** They use a very deep, heavy neural network called ResNet-50. They also try to calculate 3D depth from flat images, and they do not use active MediaPipe prompts.
- **Your Project's Novelty: Speed and Interactivity.** ResNet-50 is too heavy for a smooth web browser experience. Your novelty is using a *lightweight* CNN (converted to TFJS/ONNX) and combining it with MediaPipe.

3. "Art - 3533.pdf" (Generalized Passive Face Liveness Detection Using Region-Specific Mixture-of-Experts)

- **Workflow Comparison:** Like your passive step, they extract specific regions of the face (like the skin and eyes) to check for fake textures.
- **The Differences:** They use a complex "Mixture-of-Experts" (MoE) architecture. This means they run multiple AI models at the exact same time and have them vote on the result.
- **Your Project's Novelty: Browser Optimization.** Running multiple AI models simultaneously (MoE) is terrible for a lightweight web app. Your novelty is keeping the architecture simple: one lightweight passive CNN fused with simple MediaPipe math.

4. "Art - 3534.pdf" (Face Liveness Detection and Facial Recognition)

- **Workflow Comparison:** This is the closest to your full workflow! They capture HTML5 webcam video, use MediaPipe FaceMesh for active prompts (blinking/smiling), and use a CNN (ResNet-18) for passive texture checking.
- **The Differences:** After proving the face is real, they pass the image into another heavy AI called "FaceNet" to recognize exactly *who* the user is.
- **Your Project's Novelty: Pure Liveness Focus.** Your project strips away the heavy "facial recognition" (FaceNet) part. By focusing *only* on Liveness and converting your CNN to ONNX/TFJS, your project will run much faster on weak client devices (like cheap smartphones) without crashing the browser.

5. "Art - 3535.pdf" (SONICUMOS: An Enhanced Active Face Liveness Detection System...)

- **Workflow Comparison:** They require the user to actively perform head gestures (like nodding) to prove they are real.
- **The Differences:** Instead of using visual AI to track the head, they play an inaudible high-frequency ultrasonic sound from the phone's speaker and use the microphone like a radar to measure how the sound bounces off the user's 3D face,.
- **Your Project's Novelty: Hardware Independence.** Your project relies purely on standard computer vision (WebRTC cameras). You do not need to hack the user's speaker and microphone to emit ultrasonic sounds, which makes your web app much safer and easier to deploy.

6. "Enhancing Face Liveliness Detection Using a Depthwise Separable Convolutional Neural Network.pdf"

- **Workflow Comparison:** Matches your **OFFLINE** workflow. They use a lightweight "Depthwise Separable CNN" (like MobileNet) and artificially generate data offline to balance their dataset,.
- **The Differences:** To spot fake screens, they convert images into invisible frequencies using Fast Fourier Transform (FFT). They do not use MediaPipe.
- **Your Project's Novelty: Client-Side Processing.** Calculating Fourier frequencies on live video inside JavaScript is mathematically heavy. Your novelty is dropping the heavy math and instead using **MediaPipe active logic** to prove liveness, keeping the frame rate high.

7. "Hybrid Deep Neural Network for Face Liveness Detection in Real-Time Video.pdf"

- **Workflow Comparison:** They extract the face from the video and pass it through a CNN to classify live vs. spoof faces,.
- **The Differences:** They use a heavy face detector called MTCNN to find the face, and they use a complex dual-branch AI that analyzes spatial features and Fourier frequencies at the same time.
- **Your Project's Novelty: Frontend Speed.** Your novelty is using MediaPipe to detect the face instead of MTCNN. MediaPipe is vastly faster for browsers. You also replace their complex dual-branch frequency AI with your simple Active + Passive fusion logic.

8. "LDMRes-Net_A_Lightweight_Neural_Network_for_Efficient_Medical_Image... .pdf"

- **Workflow Comparison:** They focus entirely on building a highly lightweight, fast CNN for weak devices (IoT).
- **The Differences:** This paper is entirely about the **medical field**—specifically, segmenting blood vessels in human eyeballs to help doctors. It has nothing to do with security or spoofing.
- **Your Project's Novelty: Domain Application.** You are taking the concept of "lightweight AI for weak devices" and successfully applying it to a completely different industry: **Cybersecurity and Biometric Anti-Spoofing**.

9. "Liveness_detection_based_on_3D_face_shape_analysis.pdf"

- **Workflow Comparison:** They aim to prove the face is a real 3D object and not a flat photograph.
- **The Differences:** They require a specialized, expensive 3D scanner (optoelectronic stereo system) with multiple cameras to physically map the 3D curves of the face,.
- **Your Project's Novelty: Accessibility.** Your novelty is that you do not require users to buy expensive 3D hardware. Your system achieves high security using standard 2D webcams by smartly combining MediaPipe and CNNs.

10. "Real-Time Optimization and Lightweight Architecture of Face Detection Models for Embedded Systems.pdf"

- **Workflow Comparison:** Matches your **OFFLINE conversion** step perfectly. They trained a lightweight MobileNet model and specifically converted it to **ONNX format** to make it run incredibly fast on weak hardware,.

- **The Differences:** They built an optimized model just for "Face Detection" (drawing a box around a face). They do not check if the face is a real person (Liveness).
- **Your Project's Novelty: Liveness Focus & Web Delivery.** You are taking their brilliant offline techniques (ONNX conversion for speed) and applying them specifically to a **Liveness Classification** CNN, and delivering it directly through a browser using WebRTC/TFJS rather than a standalone Raspberry Pi.

11. "Real_Time_Liveness_Detection_and_Face_Recognition_with_OpenCV_and_Deep_Learning.pdf"

- **Workflow Comparison:** Matches your **Passive CNN** step. They crop out Regions of Interest (ROI) like the eyes and mouth and use a very simple, lightweight 3-layer CNN to classify real vs. fake,.
- **The Differences:** It is built for desktop Python/OpenCV, lacks active MediaPipe tracking, and includes a face recognition module (identifying the person).
- **Your Project's Novelty: WebRTC & Two-Layer Defense.** Your project moves the AI directly to the frontend (browser). Furthermore, because their CNN is so simple, it can be tricked; your novelty is reinforcing the simple CNN with a second layer of defense (Active MediaPipe blinks), creating a highly secure "Fusion" engine.

12. "getPDF.jsp.pdf" (Liveness Detection Based on Eye Blinks and Pose Estimation)

- **Workflow Comparison:** Matches your **ONLINE Active** step. They completely rely on MediaPipe, calculate the Eye Aspect Ratio (EAR) for blinking, and only take a picture if the user is moving,.
- **The Differences:** Because they don't have a passive CNN to check for fake paper textures, they force the user to perform complex arm poses (like raising arms to specific mathematical angles) to prove they are 3D,.
- **Your Project's Novelty: Superior User Experience (UX).** Forcing users to step back and wave their arms at specific angles just to log in is annoying. Your novelty is replacing these frustrating arm poses with your silent **Passive CNN**. The user just looks at the camera and blinks natively, and your CNN handles the rest of the security silently in the background.